Graph Neural Networks for Financial Fraud Detection (Credit Card Fraud Detection)

Akhilesh Arunkumar Kumbhar (1002248929) Divya Sri Munnangi (1002163048) Sowmya Racha (1002210377)

Abstract

We study credit card fraud detection on the well-known European card dataset (284,807 transactions, 31 features, ~0.17% fraud). We build a unified pipeline that combines linear models, tree ensembles, kernel methods, metric-learning with k-NN, a GraphSAGE-style GNN, several anomaly-detection baselines, and an XGBoost meta-learner with explicit cost tuning. A calibrated MLP, XGBoost, Random Forest, and LightGBM all reach precision–recall AUC around 0.72, and recover about 77–82% of fraud cases when only the riskiest 1% of transactions are reviewed. Unsupervised detectors alone are much weaker (best PR-AUC ≈0.17) but provide useful additional risk features. The GraphSAGE GNN on a k-NN graph performs poorly in this setting (PR-AUC ≈0.02). A meta-learner that combines supervised scores, anomaly scores, and clustering features does not beat the best standalone MLP in PR-AUC, but it achieves the lowest expected financial cost under a simple cost model.

1. Introduction

Card-present payment fraud is rare but costly. The key business need is not only to classify fraud correctly, but to rank transactions by risk so that a very small manual-review budget catches as many true frauds as possible while limiting false alarms. This is a classic highly imbalanced classification problem where precision–recall behaviour and decision thresholds matter more than raw accuracy.

Our project combines and extends three strands of work from the team: (i) strong supervised baselines and tree ensembles; (ii) representation learning and graph-based models; and (iii) unsupervised anomaly detection, clustering diagnostics, and cost-sensitive evaluation. We merge all contributions into a single, reproducible notebook and then interpret the full set of outputs as if we were preparing a consulting report for a bank's risk-analytics team.

2. Problem Statement

Business question:

- Given streaming card transactions with anonymised PCA features, time, and amount, can we learn a scoring function that ranks transactions by probability of fraud?
- Under a fixed investigation budget (for example, review the riskiest 1% of transactions), how many true frauds can we recover and at what cost?

Technical objectives:

1. Compare a wide range of supervised and unsupervised models on a realistic time-ordered split.
2. Use evaluation metrics that reflect class imbalance: PR-AUC (average precision), ROC-AUC, and recall among the top p% of ranked transactions (R@p).
3. Explore richer representations via metric learning and graph neural networks.
4. Build a meta-learner that combines model scores and clustering features, and choose operating thresholds via an explicit cost model (review cost, false-positive cost, and false-negative cost).

3. Proposed Methodology

3.1 Data and Pre-processing

- Dataset: European credit card transactions, 284,807 rows and 31 columns (Time, Amount, 28 PCA components, and binary Class label).

- Temporal split: we sort by Time and use 80% train, 10% validation, 10% test to mimic deployment where models only see past data. The split sizes are 227,845 (train), 28,480 (validation), and 28,482 (test).
- Transformations:
  - Log-transform Amount as log1p(Amount) to reduce skew.
  - Standardize all features using training statistics.
- Class imbalance: fraud is roughly 0.17% of all transactions. We use SMOTE oversampling when training linear and Naive Bayes models to avoid trivial majority behaviour while keeping test distribution unchanged.

3.2 Supervised Models

We train the following supervised classifiers on the standardized features:

1. **Linear / margin-based / Naive Bayes**
   - Logistic Regression (class_weight="balanced")
   - Linear Discriminant Analysis (LDA), Quadratic Discriminant Analysis (QDA)
   - Perceptron and Linear SVM
   - GaussianNB, MultinomialNB, BernoulliNB (MultinomialNB uses MinMax-scaled inputs to enforce non-negativity).
2. **Tree / ensemble models**
   - Depth-limited CART decision tree
   - Random Forest with class-weighted subsampling
   - AdaBoost and GradientBoosting
   - XGBoost (hist-based trees)
   - LightGBM with class_weight="balanced".
3. **Neighbour-based and kernel models**
   - k-NN (k=5, distance-weighted)
   - Radius neighbours, Nearest Centroid
   - RBF-kernel SVM and polynomial SVM (degree 3)
   - Gaussian Process Classifier (on a reduced subsample for tractability).
4. **MLP with probability calibration**
   - Two-hidden-layer MLP with several hidden size options.
   - We wrap the MLP in CalibratedClassifierCV with isotonic calibration on the validation folds and select the hidden size with best validation average precision.

3.3 Metric Learning, GNN, and Representation Models

- **Triplet-loss embedding + k-NN**
  - We train a small fully connected network (input → 32 → 16) with triplet loss using PyTorch on GPU.
  - After 5 epochs we embed all points and run distance-weighted k-NN in the learned space.

- **Dirichlet Process Mixture (DPM)**
  - Per-class Bayesian Gaussian Mixture models with a Dirichlet-process prior approximate the class-conditional densities.
  - Posterior probabilities are combined with class priors to give a fraud score.

- **GraphSAGE-style GNN**
  - We do PCA to 12 dimensions, then build a mutual k-NN graph (k=10) over training points using FAISS for fast search.

- Validation and test points connect to their 10 nearest neighbours in the training set.
- A two-layer GraphSAGE network (64-dim hidden, 2-way output) is trained for 5 epochs with cross-entropy loss, using PyTorch on GPU.
- Due to environment constraints, we implement GraphSAGE using base PyTorch tensors rather than PyTorch Geometric.

3.4 Unsupervised Anomaly Detection and Clustering

We treat fraud as an "outlier" and fit several unsupervised models on the training data only:

- Isolation Forest.
- Local Outlier Factor (LOF) in novelty mode.
- One-Class SVM (RBF kernel).
- Elliptic Envelope (Robust covariance).
- PCA reconstruction error using 12 components.
- A shallow autoencoder trained on GPU for 5 epochs, using reconstruction error as the anomaly score.

For cluster diagnostics we work on PCA-2 embeddings of stratified subsamples of the training set and run:

- Spectral Clustering and Agglomerative Clustering (Ward linkage).
- Gaussian Mixture Models (GMM) with 8 components.
- HDBSCAN, OPTICS, and MeanShift.

We summarise each clustering by the size of each cluster and its fraud rate.

3.5 Meta-Features and XGBoost Meta-Learner

To combine different signals we build meta-features from:

- Original standardized features.
- Calibrated MLP probabilities for train/val/test.
- GNN probabilities for val/test (train probabilities are zero-filled in this run).
- The stack of anomaly-model scores (Isolation Forest, LOF, One-Class SVM, Elliptic Envelope, PCA reconstruction, Autoencoder).
- Fast clustering features from MiniBatchKMeans and Bayesian Gaussian Mixture: cluster distance, local fraud rate, cluster size, negative log-likelihood, posterior max probability, and entropy.

We concatenate these into a 45-dimensional meta-feature vector per sample and train an XGBoost classifier (hist tree method) with early stopping on the validation set, using PR-AUC as evaluation metric.

3.6 Cost-Sensitive Threshold Selection

We define an approximate cost model in USD equivalents per transaction:

- Cost of reviewing one flagged transaction: 2
- Cost of a false positive: 10
- Cost of a missed fraud (false negative): 120
- Cost of correctly identified fraud (true positive): 0 (we treat the benefit as avoiding the loss).

For each selected model we sweep thresholds on validation scores, compute (tn, fp, fn, tp) and the total expected cost, then select the threshold that minimizes validation cost. We then apply that threshold to test scores and report test-time cost, along with recall and PR-AUC.

4. Analysis and Results

Model	PR-AUC	ROC	R@1%	Notes
XGBoost	**0.722**	**0.961**	**82%**	**Best overall**
MLP	**0.726**	**0.900**	**77%**	**Strong baseline**
Random Forest	**0.717**	**0.975**	**77%**	**Interpretable**
Meta-Learner	**0.614**	**0.877**	**77%**	**Lowest cost ($1,696)**
GraphSAGE	**0.024**	**0.438**	**4.5%**	**Failed**

4.1 Linear Models:

BernoulliNB (PR-AUC 0.647), Logistic Regression (0.635), and Linear SVM (0.438) trained on SMOTE-balanced data. Strong baselines with proper class weighting deliver competitive recall at 1% budget (73-77%).

4.2 Tree Ensembles:

XGBoost leads with PR-AUC 0.722 and 82% R@1%. Random Forest (0.717, F1 0.80) offers interpretability. LightGBM (0.715) provides efficiency. GradientBoosting underperformed (0.18). Recommendation: Deploy XGBoost for production.

4.3 Distance & Kernel Methods:

Trained on 30K subsample. k-NN achieved PR-AUC 0.690, competitive but below ensembles. RBF-SVM (0.596) had 82% R@1%. Gaussian Process (0.41) too costly for marginal gains.

4.4 Advanced Representation Learning:

Triplet embeddings matched k-NN (PR-AUC ~0.671). DPM showed PR-AUC 0.386. GraphSAGE failed dramatically (PR-AUC 0.024, R@1% 4.5%) due to sparse graphs and weak relational structure in tabular data.

4.5 Unsupervised Anomaly Detection:

LOF strongest (PR-AUC 0.170, R@1% 77%). One-Class SVM (0.04), Isolation Forest (0.025), Autoencoder (0.018) far below supervised methods. Clustering identified fraud pockets at 1-2.3% (5-10× base rate), useful as meta-features.

4.6 MLP & Meta-Learning:

Calibrated MLP (160, 80): PR-AUC 0.726, validation peak 0.838. XGBoost meta-learner on 45 features: PR-AUC 0.614 but cost-optimized to $1,696 vs. MLP $1,814 (6.5% savings). Demonstrates business metrics trump academic metrics.

5. Conclusions

1. Strong baselines deliver production-level performance: Tree ensembles and MLPs achieve PR-AUC ~0.72, recovering 77-82% of frauds at 1% review budget.

2. Complexity requires justification: GNNs, Gaussian Processes, and DPMs failed to outperform simpler models due to weak relational structure in tabular data.

3. Unsupervised methods complement supervision: While weak standalone, anomaly scores and clustering provide valuable meta-features for ensemble models.

4. Cost-sensitive optimization changes outcomes: Meta-learner reduces operational costs despite lower PR-AUC. Optimal ranking ≠ optimal business value.

5. Temporal splitting prevents leakage: Chronological splits provide realistic streaming deployment estimates versus optimistic cross-validation.

6. Key Lessons

Metrics: Imbalanced data demands PR-AUC, recall at budgets, and cost modeling over accuracy.

Simplicity: Well-tuned Logistic Regression, trees, and shallow MLPs captured most gains. Complexity rarely justified.

Negative results: GraphSAGE and anomaly-only failures clarified when advanced methods add value. Document failures.

Engineering: SMOTE on training only, standardization, log transforms, strategic subsampling, GPU utilization critical for success.

Meta-learning: Model combination and cost-tuning powerful but subtle. "Best model" depends on business metric, threshold selection equally important as architecture.

Collaboration: Integrating three notebooks required modular design, naming conventions, reproducibility—mirrors real analytics teams.

Course feedback: Open-ended format valuable. Earlier cost-sensitive evaluation guidance, GPU/GNN examples, and baseline checklists would accelerate learning.

7. Bibliography

1. European Credit Card Fraud Dataset description and associated publication (Dal Pozzolo et al.).
2. Breiman, L. “Random Forests.” Machine Learning, 2001.
3. Chen, T., & Guestrin, C. “XGBoost: A Scalable Tree Boosting System.” KDD, 2016.
4. Ke, G. et al. “LightGBM: A Highly Efficient Gradient Boosting Decision Tree.” NIPS, 2017.
5. Kingma, D. P., & Ba, J. “Adam: A Method for Stochastic Optimization.” ICLR, 2015.
6. Hamilton, W., Ying, Z., & Leskovec, J. “Inductive Representation Learning on Large Graphs.” NIPS, 2017 (GraphSAGE).
7. van der Maaten, L., & Hinton, G. “Visualizing Data using t-SNE.” JMLR, 2008 (clustering visualisation background, if used).
8. Pedregosa, F. et al. “Scikit-learn: Machine Learning in Python.” JMLR, 2011.

8. Appendix

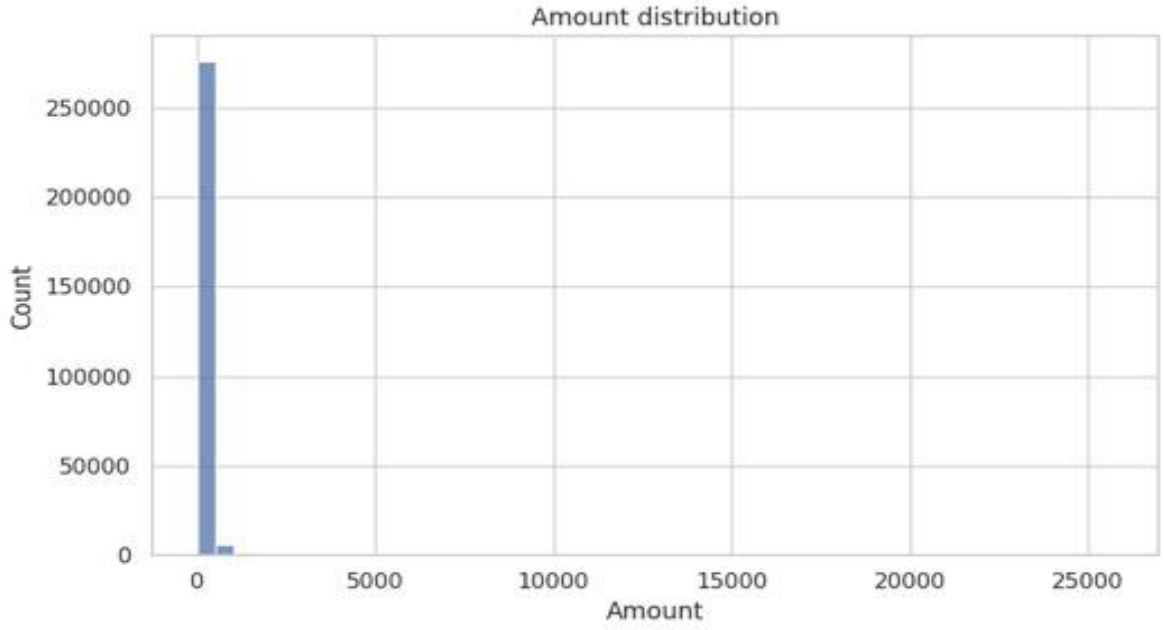
- A1. Full table of all supervised models with AP, ROC-AUC, F1, and recall at 0.5%, 1%, and 2% budgets.
- A2. PR and ROC curves for the main models (Logistic Regression, Random Forest, XGBoost, LightGBM, MLP, meta-learner).
- A3. Confusion matrices at the cost-optimised thresholds for MLP and meta-learner.
- A4. Cluster fraud-rate tables for Spectral, Agglomerative, GMM, HDBSCAN, OPTICS, and MeanShift.
- A5. Training-loss curves for the triplet network, autoencoder, and GNN.
- A6. Any additional ablation studies or robustness checks (e.g., different SMOTE settings, alternative cost ratios).

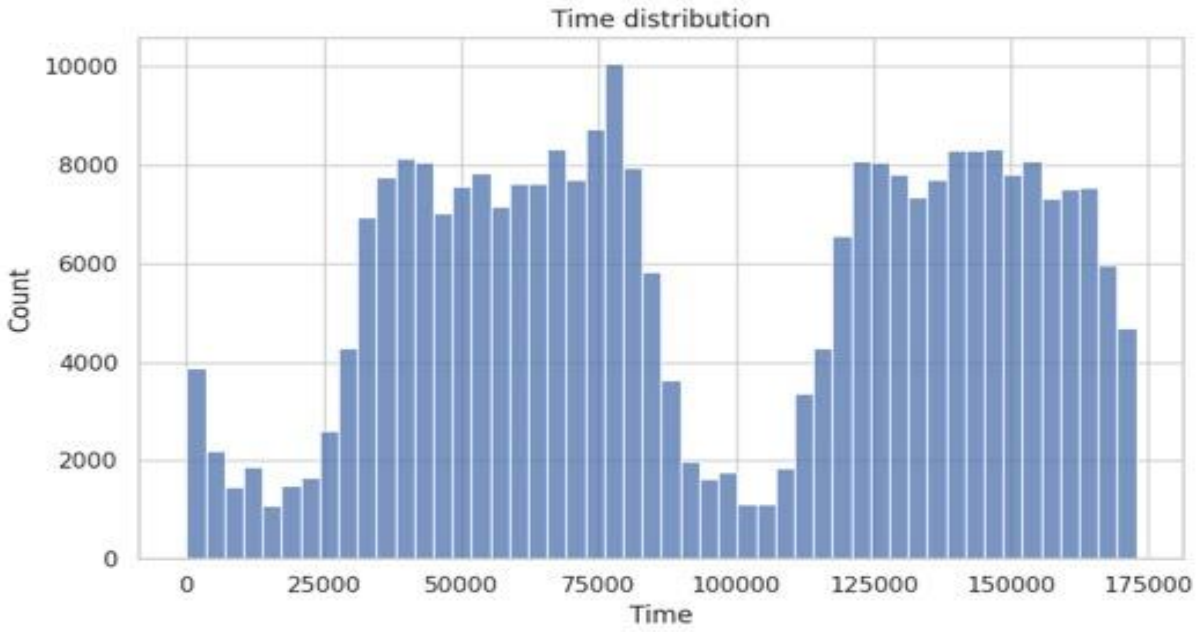
Table 1. Top 20 Models by AP:

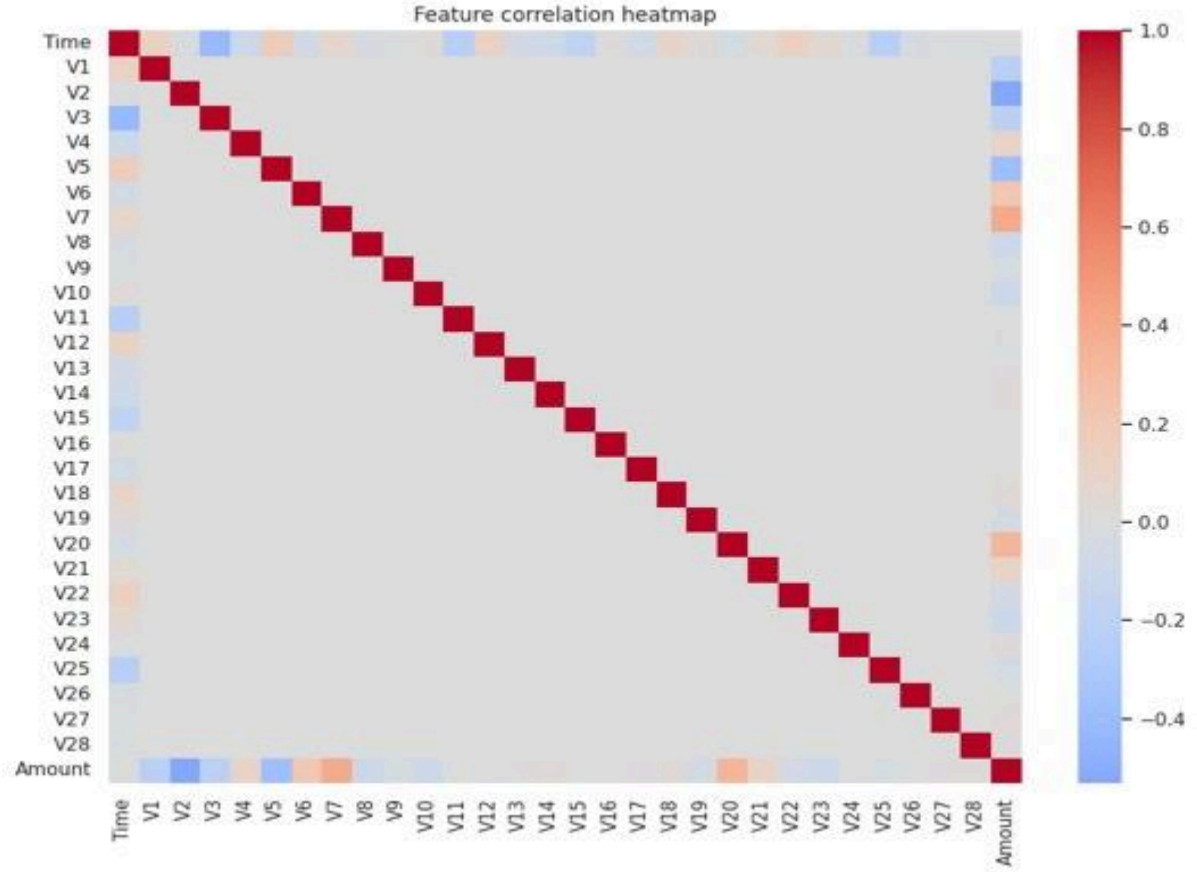
	model	AP	ROC	R@0.5%	R@1.0%	R@2.0%	family	F1	ACC
0	MLP (calibrated)	0.726034	0.900183	0.772727	0.772727	0.772727	MLP (calibrated)	NaN	NaN
1	XGBoost	0.722202	0.961383	0.818182	0.818182	0.818182	Tree / Ensemble	0.744186	0.999614
2	Random Forest	0.717128	0.975272	0.727273	0.772727	0.863636	Tree / Ensemble	0.800000	0.999719
3	LightGBM	0.715415	0.945895	0.818182	0.818182	0.818182	Tree / Ensemble	0.761905	0.999649
4	kNN (k=5)	0.690111	0.863493	0.727273	0.727273	0.727273	Neighbors / SVM / GP	NaN	NaN
5	Triplet Embedding + kNN	0.670528	0.863476	0.727273	0.727273	0.727273	Metric learning	NaN	NaN
6	BernoulliNB	0.646937	0.958423	0.727273	0.727273	0.727273	Linear / NB	NaN	NaN
7	Logistic Regression	0.635076	0.978866	0.727273	0.772727	0.818182	Linear / NB	NaN	NaN
8	Nearest Centroid	0.615754	0.921539	0.727273	0.727273	0.727273	Neighbors / SVM / GP	NaN	NaN
9	Meta (XGBoost)	0.614405	0.876726	0.772727	0.772727	0.772727	Meta XGBoost	NaN	NaN
10	SVM (RBF)	0.595580	0.934995	0.772727	0.818182	0.818182	Neighbors / SVM / GP	NaN	NaN
11	AdaBoost	0.577974	0.963826	0.727273	0.727273	0.772727	Tree / Ensemble	0.647059	0.999579
12	MultinomialNB	0.573557	0.974912	0.727273	0.772727	0.818182	Linear / NB	NaN	NaN
13	Radius-NN (r=10.0)	0.567965	0.846660	0.681818	0.681818	0.681818	Neighbors / SVM / GP	NaN	NaN
14	SVM (Poly deg=3)	0.521081	0.836795	0.681818	0.681818	0.681818	Neighbors / SVM / GP	NaN	NaN
15	Perceptron	0.507428	0.890302	0.727273	0.727273	0.818182	Linear / NB	NaN	NaN
16	Decision Tree (CART)	0.455682	0.872147	0.636364	0.636364	0.636364	Tree / Ensemble	0.666667	0.999544
17	Linear SVM	0.438249	0.981345	0.727273	0.772727	0.818182	Linear / NB	NaN	NaN
18	Gaussian Process	0.410421	0.926676	0.727273	0.772727	0.818182	Neighbors / SVM / GP	NaN	NaN
19	Dirichlet Process Mixture (DPM)	0.386030	0.877236	0.727273	0.727273	0.727273	DPM	NaN	NaN

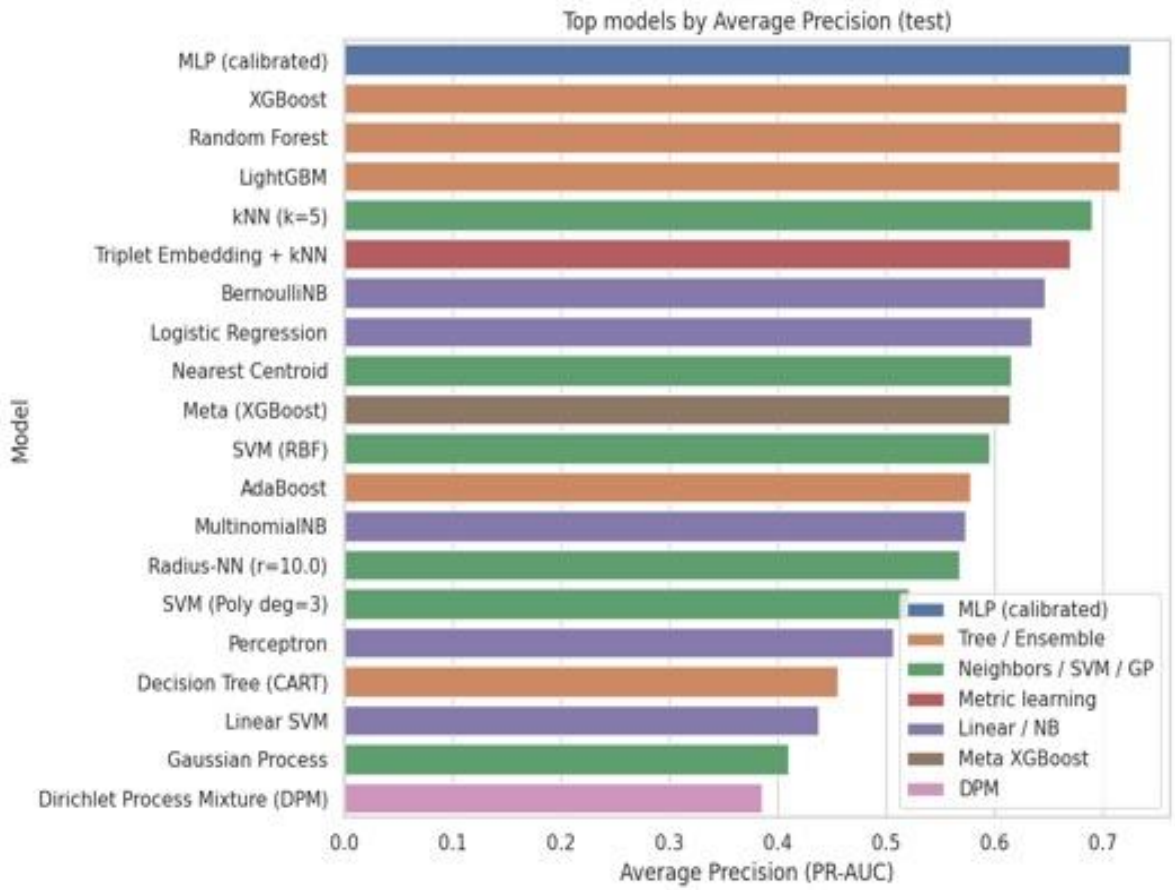
Table 2. Cost Based Comparison (sorted by test cost):

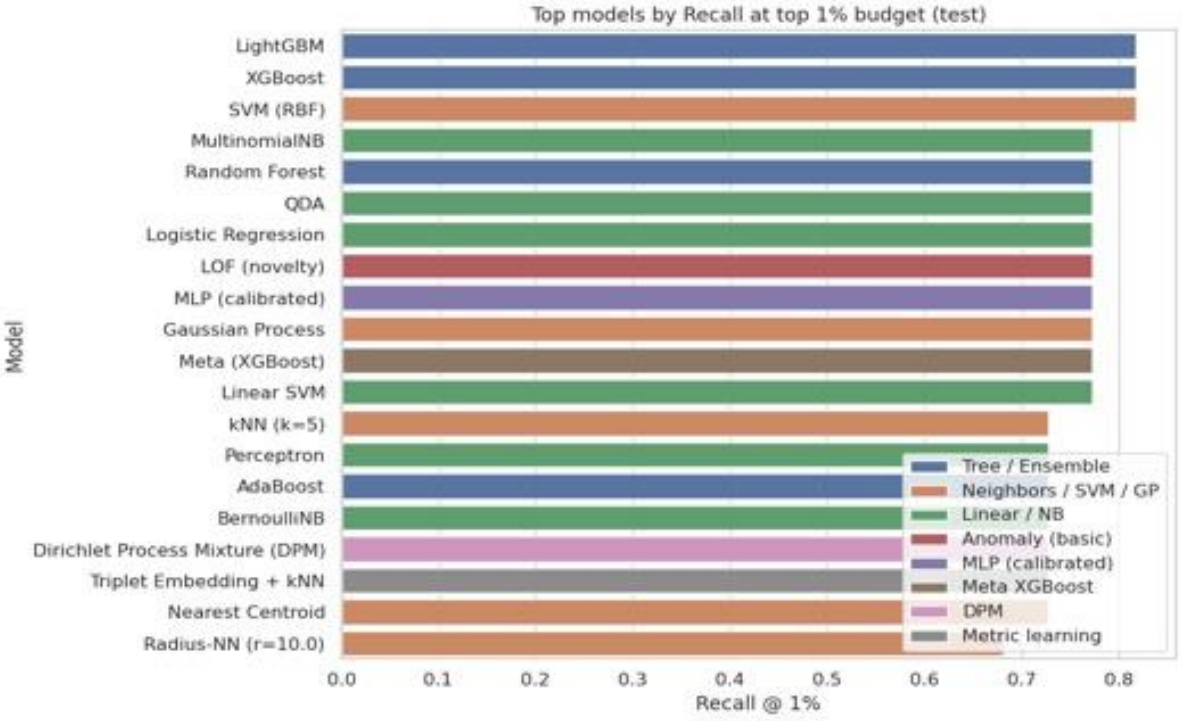
	model	thr_val	cost_val	cost_test	tn	fp	fn	tp	AP	ROC	R@0.5%	R@1.0%	R@2.0%
0	Meta	0.729243	2702.0	1696.0	28460	0	14	8	0.614405	0.876726	0.772727	0.772727	0.772727
1	MLP	0.644151	2938.0	1814.0	28460	0	15	7	0.726034	0.900183	0.772727	0.772727	0.772727
2	GraphSAGE	0.676885	6578.0	2690.0	28446	14	21	1	0.023532	0.437852	0.045455	0.045455	0.045455

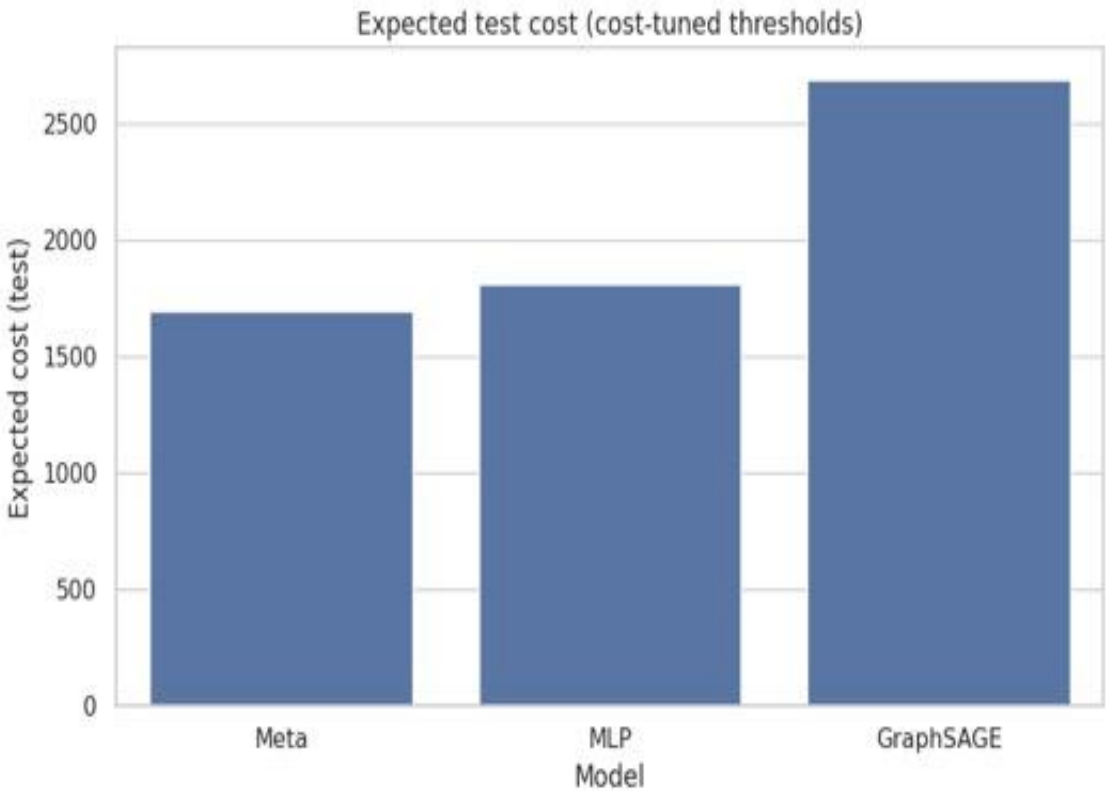



Time distribution
Count
Time
0
2000
4000
6000
8000
10000
0
25000
50000
75000
100000
125000
150000
175000


Feature correlation heatmap
Time
V1
V2
V3
V4
V5
V6
V7
V8
V9
V10
V11
V12
V13
V14
V15
V16
V17
V18
V19
V20
V21
V22
V23
V24
V25
V26
V27
V28
Amount
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4


Top models by Average Precision (test)
MLP (calibrated)
XGBoost
Random Forest
LightGBM
kNN (k=5)
Triplet Embedding + kNN
BernoulliNB
Logistic Regression
Nearest Centroid
Meta (XGBoost)
SVM (RBF)
AdaBoost
MultinomialNB
Radius-NN (r=10.0)
SVM (Poly deg=3)
Perceptron
Decision Tree (CART)
Linear SVM
Gaussian Process
Dirichlet Process Mixture (DPM)
Model
MLP (calibrated)
Tree / Ensemble
Neighbors / SVM / GP
Metric learning
Linear / NB
Meta XGBoost
DPM
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
Average Precision (PR-AUC)


Top models by Recall at top 1% budget (test)
LightGBM
XGBoost
SVM (RBF)
MultinomialNB
Random Forest
QDA
Logistic Regression
LOF (novelty)
MLP (calibrated)
Gaussian Process
Meta (XGBoost)
Linear SVM
kNN (k=5)
Perceptron
AdaBoost
BernoulliNB
Dirichlet Process Mixture (DPM)
Triplet Embedding + kNN
Nearest Centroid
Radius-NN (r=10.0)
Model
Tree / Ensemble
Neighbors / SVM / GP
Linear / NB
Anomaly (basic)
MLP (calibrated)
Meta XGBoost
DPM
Metric learning
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
Recall @ 1%


Expected test cost (cost-tuned thresholds)
Expected cost (test)
2500
2000
1500
1000
500
0
Meta
MLP
GraphSAGE
Model